

UD3.1: Debugging con VSCode — Guía práctica

Módulo: ESP — Entornos y Sintaxis en Python (5098)

Unidad: UD3 — Entornos y Control de Versiones

Criterios: CE2_5 (errores frecuentes), CE2_6 (importancia de depuración), CE2_7 (análisis antes/después de depuración)

Índice de contenidos

- 1. Tipos de errores en Python
- 2. Leer un traceback
- 3. Print debugging (técnica rápida)
- 4. El debugger de VSCode
- 5. Ejercicio resuelto 1: Breakpoints y Step Over
- 6. Ejercicio resuelto 2: Step Into y Step Out
- 7. Ejercicio resuelto 3: Watch expressions
- 8. Ejercicio resuelto 4: Breakpoints condicionales
- 9. Ejercicio resuelto 5: Call Stack

Archivo de ejercicios: los programas de los ejercicios resueltos 1–5 están disponibles en `debug_demo.py` . Abrir ese archivo en VSCode para seguir los ejercicios con el debugger.

1. Tipos de errores en Python

Existen tres tipos de errores en un programa:

Tipo	¿Python avisa?	Ejemplo	¿Cómo se encuentra?
SyntaxError	Sí (no arranca)	<code>if x > 5 sin :</code>	Leer el mensaje de error

Tipo	¿Python avisa?	Ejemplo	¿Cómo se encuentra?
Excepción	Sí (traceback)	lista[100]	Leer el traceback (sección 2)
Error lógico	No	n % 2 == 1 para pares	Print (sección 3) o debugger (sección 4)

Errores de sintaxis

Python detecta que algo está mal escrito y **no puede arrancar**. Indica la línea y el punto exacto con ^ :

```
if x > 5
    print("mayor")
# SyntaxError: expected ':'
```

Excepciones (errores en tiempo de ejecución)

El programa arranca pero **se rompe** durante la ejecución. Las más comunes:

Excepción	Causa habitual
NameError	Variable no definida (typo): print(nombr)
TypeError	Operación con tipo incorrecto: "hola" + 5
IndexError	Índice fuera de rango: lista[100]
KeyError	Clave no existe en diccionario
ZeroDivisionError	División entre cero: 10 / 0
ValueError	Valor incorrecto: int("hola")
AttributeError	Método/atributo no existe: "hola".appnd("!")

Errores lógicos

El programa se ejecuta sin errores, pero **produce un resultado incorrecto**. Python no emite ningún aviso.

```
def es_par(n):
    return n % 2 == 1 # Bug: == 1 comprueba IMPAR, no par

print(es_par(4)) # False (debería ser True)
```

Para encontrar errores lógicos se necesitan técnicas de debugging: `print()` o el debugger de VSCode.

2. Leer un traceback

Cuando se produce una excepción, Python muestra un **traceback**. Se lee **de abajo a arriba**:

```
Traceback (most recent call last):
  File "pedido.py", line 13, in <module>
    print(procesar_pedido(precios, descuentos))
    ~~~~~
  File "pedido.py", line 8, in procesar_pedido
    total += calcular_precio(productos[i], descuentos[i])
                                ~~~~~^
IndexError: list index out of range
```

Paso	Dónde mirar	Qué dice
1	Última línea	Tipo de error: <code>IndexError: list index out of range</code>
2	Línea anterior	Código que falló: <code>descuentos[i]</code>
3	"File..."	Archivo y línea: <code>pedido.py, line 8</code>
4	Secciones superiores	Cadena de llamadas que llevó hasta ahí

Regla: El traceback muestra **dónde se manifestó** el error, no siempre **dónde se originó**. El `IndexError` se manifiesta en la línea 8, pero el problema real puede estar en los datos que se pasan a la función.

3. Print debugging

La técnica más básica: insertar `print()` para ver los valores de las variables en cada momento.

Ejemplo

```
def media_positivos(numeros):
    suma = 0
    contador = 0
    for num in numeros:
        if num > 0:
            suma += num
            contador += 1
        print(f" num={num}, suma={suma}, contador={contador}") # ← DEBUG
    return suma / contador

datos = [5, -3, 8, -1, 4, -2, 7]
print(media_positivos(datos)) # Esperado: 6.0, Obtenido: 3.43
```

Salida:

```
num=5, suma=5, contador=1
num=-3, suma=5, contador=2      ← El contador sube con -3
num=8, suma=13, contador=3
num=-1, suma=13, contador=4    ← Y con -1
...
```

El `contador += 1` está **fuera** del `if` : cuenta todos los números, no solo los positivos.

Tips

- Usar **f-strings descriptivas**: `print(f"suma={suma}, contador={contador}")` en vez de `print(suma)`
- Marcar la **ubicación**: `print(f"[ANTES del if] nota={nota}")`
- Usar `repr()` para ver **espacios ocultos**: `print(repr(nombre))` muestra `'Ana '`
- **Eliminar** los prints al terminar

Limitaciones

- Hay que **modificar el código** (y después limpiarlo)

- En programas grandes se vuelve inmanejable
- No permite **pausar** la ejecución para explorar el estado completo

Para superar estas limitaciones, usamos el debugger de VSCode.

4. El debugger de VSCode

El debugger permite **pausar la ejecución**, avanzar **paso a paso** e inspeccionar **todas las variables** sin modificar el código fuente.

4.1. Breakpoints

Un **breakpoint** (punto de parada) indica al debugger que debe detener la ejecución en esa línea.

Cómo ponerlo: hacer clic en el **margen izquierdo**, junto al número de línea. Aparece un **punto rojo**.

```
1 | def media_positivos(numeros):
2 |     suma = 0
3 |     contador = 0
4 |     for num in numeros:      ← breakpoint aquí
5 |         if num > 0:
6 |             suma += num
7 |             contador += 1
8 |     return suma / contador
```

Para quitarlo: hacer clic de nuevo en el punto rojo. Se pueden poner **varios** a la vez.

4.2. Iniciar el debugger

- Pulsar **F5** (o menú Run → Start Debugging)
- Seleccionar **"Python File"** si lo pregunta
- El programa arranca y **se detiene** en el primer breakpoint
- La línea actual se resalta en **amarillo**

4.3. Controles del debugger

Aparece una barra flotante en la parte superior:

Tecla	Nombre	Qué hace
F10	Step Over	Ejecuta la línea actual y pasa a la siguiente
F11	Step Into	Si hay una llamada a función, entra dentro
Shift+F11	Step Out	Sale de la función actual y vuelve a quien la llamó
F5	Continue	Sigue ejecutando hasta el próximo breakpoint (o hasta el final)
Ctrl+Shift+F5	Restart	Reinicia el programa desde el principio
Shift+F5	Stop	Detiene la ejecución

4.4. Paneles de información

Panel "Variables"

Muestra **todas las variables** y sus valores en el contexto actual. Se actualiza automáticamente a cada paso.

▼ Locals

```

numeros = [5, -3, 8, -1, 4, -2, 7]
suma = 5
contador = 1
num = -3

```

Panel "Watch" (Observar)

Permite añadir **expresiones personalizadas** para vigilar. Hacer clic en + y escribir cualquier expresión Python:

Watch:

```

suma / contador    → 5.0
num > 0            → False
len(numeros)       → 7

```

Útil para vigilar condiciones o cálculos intermedios sin tocar el código.

Panel "Call Stack" (Pila de llamadas)

Muestra la **cadena de funciones** que ha llevado hasta el punto actual. Al hacer clic en cada nivel, el panel Variables cambia para mostrar las variables de esa función.

Hover

Mientras el programa está pausado, al pasar el ratón por encima de cualquier variable en el código se muestra su valor actual.

5. Ejercicio resuelto 1: Breakpoints y Step Over

Se aprende: poner breakpoints, F10 (Step Over), panel Variables, hover

El programa

```
# EJERCICIO RESUELTO 1 – debug_demo.py
def media_positivos(numeros):
    """Calcula la media de los números positivos."""
    suma = 0
    contador = 0
    for num in numeros:
        if num > 0:
            suma += num
            contador += 1
    return suma / contador

datos = [5, -3, 8, -1, 4, -2, 7]
print(f"Media positivos: {media_positivos(datos)}")
# Esperado: 6.0
# Obtenido: 3.43
```

Qué hacer

1. Abrir `debug_demo.py` en VSCode
2. Poner un **breakpoint** en la línea `for num in numeros:` (clic en el margen)
3. Pulsar **F5** para iniciar el debugger
4. El programa se detiene en el breakpoint. Mirar el panel **Variables**:
 - `numeros = [5, -3, 8, -1, 4, -2, 7]`

- `suma = 0`
- `contador = 0`

5. Pulsar **F10** → entra en el bucle. `num = 5`
6. Pulsar **F10** → evalúa `if num > 0` : es `True`
7. Pulsar **F10** → ejecuta `suma += num` . Verificar en Variables: `suma = 5`
8. Pulsar **F10** → ejecuta `contador += 1` . Verificar: `contador = 1`
9. Pulsar **F10** → vuelve al `for` . `num = -3`
10. Pulsar **F10** → evalúa `if num > 0` : es `False` , **salta** el bloque del `if`
11. Pulsar **F10** → ejecuta `contador += 1` . Verificar: `contador = 2`

Bug encontrado

El `contador` subió a 2, pero -3 no es un número positivo. La línea `contador += 1` está **fuera** del `if` : cuenta todos los números, no solo los positivos.

Corrección

```
if num > 0:
    suma += num
    contador += 1 # ← dentro del if
```

6. Ejercicio resuelto 2: Step Into y Step Out

Se aprende: F11 (Step Into), Shift+F11 (Step Out), diferencia entre F10 y F11

El programa

```
# EJERCICIO RESUELTO 2 – debug_demo.py
def aplicar_descuento(precio, porcentaje):
    """Aplica un descuento porcentual a un precio."""
    descuento = precio * porcentaje
    precio_final = precio - descuento
    return precio_final

precios = [50, 30, 80]
total = 0
for p in precios:
    total += aplicar_descuento(p, 10)
print(f"Total con 10% descuento: {total}")
# Esperado: 144.0
# Obtenido: ???
```

Qué hacer

1. Poner un breakpoint en la línea `total += aplicar_descuento(p, 10)`
2. Pulsar **F5**. El programa se detiene en el breakpoint
3. Pulsar **F10** (Step Over) → la línea se ejecuta completa. Mirar Variables: `total = -450.0`
 - Con F10, la función se ejecutó entera sin entrar en ella. Algo va mal dentro, pero no se puede ver qué
4. Pulsar **Ctrl+Shift+F5** (Restart) para reiniciar
5. Esta vez, pulsar **F11** (Step Into) → el debugger **entra dentro** de `aplicar_descuento`
6. Mirar Variables: `precio = 50` , `porcentaje = 10`
7. Pulsar **F10** → ejecuta `descuento = precio * porcentaje` . Variables: `descuento = 500`
 - El descuento debería ser el 10% de 50 = 5, pero sale 500. **¡Aquí está el bug!**
8. Pulsar **Shift+F11** (Step Out) → sale de la función y vuelve al bucle

Diferencia entre F10 y F11

Tecla	Sobre la línea <code>total += aplicar_descuento(p, 10)</code>
F10 (Step Over)	Ejecuta la función entera y pasa a la siguiente línea
F11 (Step Into)	Entra dentro de <code>aplicar_descuento</code> para ver paso a paso

Regla práctica: usar F10 para avanzar por el código general, F11 cuando el bug puede estar **dentro** de una función.

Corrección

```
descuento = precio * porcentaje / 100 # ← faltaba / 100
```

7. Ejercicio resuelto 3: Watch expressions

Se aprende: panel Watch, añadir expresiones personalizadas, detectar bucles que terminan antes de tiempo

El programa

```
# EJERCICIO RESUELTO 3 — debug_demo.py
def es_primo(n):
    """Devuelve True si n es primo."""
    if n < 2:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

print(f"es_primo(2) = {es_primo(2)}")    # True ✓
print(f"es_primo(4) = {es_primo(4)}")    # False ✓
print(f"es_primo(9) = {es_primo(9)}")    # Esperado: False → Obtenido: True ✗
```

Qué hacer

1. Poner un breakpoint en la línea `for i in range(2, n):`
2. En el panel **Watch**, hacer clic en `+` y añadir estas expresiones:
 - `n`
 - `i`
 - `n % i`
3. Pulsar **F5**. Se detiene en la primera llamada `es_primo(2)`
4. Pulsar **F5** de nuevo para saltar a `es_primo(9)` (la que falla)

5. El debugger se detiene en el `for` . Watch muestra: `n = 9`
6. Pulsar **F10** → `i = 2` . Watch muestra: `n % i = 1`
7. Pulsar **F10** → evalúa `if n % i == 0` : es `False` (`1 ≠ 0`)
8. Pulsar **F10** → ejecuta `return True` → ¡la función termina!

Bug encontrado

Watch revela que `i` nunca pasa de 2. El `return True` está **dentro** del `for` (mismo nivel de indentación que el `if`). El bucle solo comprueba el primer divisor (`i=2`) y si no es divisible, devuelve `True` inmediatamente. **Nunca llega a probar `i=3`** (donde `9 % 3 == 0` descubriría que no es primo).

Para qué sirve Watch

- Vigilar expresiones que **no son variables** directas (`n % i` , `suma / contador` , `len(lista) - 1`)
- Comprobar **condiciones** en cada paso (`num > 0` , `i < len(lista)`)
- Detectar que un valor **no cambia** cuando debería (como `i` que nunca pasa de 2)

Corrección

```
for i in range(2, n):
    if n % i == 0:
        return False
return True # ← FUERA del for (un nivel menos de indentación)
```

8. Ejercicio resuelto 4: Breakpoints condicionales

Se aprende: breakpoints con condición, depuración de casos específicos sin avanzar manualmente

El programa

```
# EJERCICIO RESUELTO 4 – debug_demo.py
def clasificar_temperaturas(temps):
    """Clasifica temperaturas: <0=congelación, 0-15=frío, 16-30=templado, >30=calor."""
    resultado = []
    for t in temps:
        if t < 0:
            resultado.append("congelación")
        elif t <= 15:
            resultado.append("frío")
        elif t < 30:
            resultado.append("templado")
        else:
            resultado.append("calor")
    return resultado

temperaturas = [5, -3, 22, 30, 15, 35, 0, 28, 31]
resultado = clasificar_temperaturas(temperaturas)
for t, cat in zip(temperaturas, resultado):
    print(f" {t}°C → {cat}")
# El 30°C debería ser "templado", pero aparece como "calor"
```

El problema

El programa tiene 9 temperaturas y el bug solo se manifiesta con el valor 30. Poner un breakpoint normal en el bucle obligaría a pulsar F10 muchas veces para llegar a esa iteración. Los **breakpoints condicionales** permiten detenerse solo cuando una condición se cumple.

Qué hacer

1. Poner un breakpoint en la línea `if t < 0:`
2. **Clic derecho** sobre el punto rojo → **"Edit Condition..."** (o "Editar condición...")
3. Escribir la condición: `t == 30`
4. Pulsar Enter → el punto rojo cambia de aspecto (puede mostrar un `=` o cambiar de color)
5. Pulsar **F5** → el programa se ejecuta y **solo se detiene cuando t vale 30**
6. Mirar Variables: `t = 30`
7. Pulsar **F10** → evalúa `if t < 0`: False
8. Pulsar **F10** → evalúa `elif t <= 15`: False
9. Pulsar **F10** → evalúa `elif t < 30`: False ← ¡30 **no** es menor que 30!

10. Pulsar **F10** → entra en `else` → clasifica como `"calor"`

Bug encontrado

La condición `t < 30` excluye exactamente el valor 30. Debería ser `t <= 30`.

Cuándo usar breakpoints condicionales

- Bucles con muchas iteraciones donde el bug solo aparece con **un dato concreto**
- Cuando se sabe cuál es el **valor problemático** pero no se quiere avanzar manualmente
- Para depurar programas que procesan **listas largas** (cientos de elementos)

Otros tipos de condiciones

Además de expresiones booleanas, se pueden usar:

- **Hit Count** (número de pasadas): detenerse en la 5.^a iteración → 5
- **Log Message**: imprimir un mensaje en la consola sin detener la ejecución (como un `print` invisible)

Acceder a estas opciones haciendo clic derecho en el breakpoint → "Edit Condition..." → seleccionar el tipo en el desplegable.

Corrección

```
elif t <= 30: # ← <= en vez de <
```

9. Ejercicio resuelto 5: Call Stack

Se aprende: panel Call Stack, navegar entre contextos de función, rastrear el origen de un valor incorrecto

El programa

```
# EJERCICIO RESUELTO 5 – debug_demo.py
def validar_nota(nota):
    """Devuelve True si la nota está entre 0 y 10."""
    return nota >= 0 and nota <= 10

def calcular_media(notas):
    """Calcula la media de las notas válidas."""
    validas = [n for n in notas if validar_nota(n)]
    return sum(validas) / len(notas)

def resumen_clase(notas):
    """Devuelve la media y el número de aprobados (notas válidas >= 5)."""
    media = calcular_media(notas)
    aprobados = len([n for n in notas if validar_nota(n) and n >= 5])
    return media, aprobados

notas = [7, 8, -1, 5, 15, 10, 6]
media, aprobados = resumen_clase(notas)
print(f"Media: {media:.1f}, Aprobados: {aprobados}")
# Notas válidas: [7, 8, 5, 10, 6] → media debería ser 7.2
# Esperado: Media: 7.2, Aprobados: 5
# Obtenido: ???
```

Qué hacer

1. Poner un breakpoint en `return sum(validas) / len(notas)` (dentro de `calcular_media`)
2. Pulsar **F5**. El programa se detiene
3. Mirar el panel **Call Stack**:

```
calcular_media    ← estamos aquí
resumen_clase
<module>
```

4. Mirar Variables: `validas = [7, 8, 5, 10, 6]` (5 elementos, correcto)
5. Pasar el ratón (hover) por `len(notas)` → muestra 7 (tiene 7 notas en total)
6. Pasar el ratón por `sum(validas)` → muestra 36
7. La operación será: $36 / 7 = 5.14$ → Incorrecto. Debería ser $36 / 5 = 7.2$

Bug encontrado

Se divide por `len(notas)` (todas las notas, incluidas las inválidas -1 y 15) en vez de `len(validas)` (solo las notas válidas).

Navegar el Call Stack

- 8. Hacer clic en **"resumen_clase"** en el Call Stack → el panel Variables cambia al contexto de esa función. Se puede ver que `notas` original tiene 7 elementos
- 9. Hacer clic en `""` → se ven las variables globales

Para qué sirve el Call Stack

- Ver la **cadena de llamadas** que llevó al punto actual
- **Navegar entre contextos:** al hacer clic en cada nivel, Variables muestra las variables de esa función
- Entender **cómo llegó** un valor incorrecto a una función (rastrear hacia arriba)

Corrección

```
return sum(validas) / len(validas) # ← len(validas), no len(notas)
```

Resumen

Técnicas de debugging

Técnica	Ventajas	Inconvenientes
<code>print()</code>	Rápida, no necesita herramientas	Hay que modificar y limpiar el código
Debugger VSCode	No modifica código, potente, visual	Requiere aprender los controles

Atajos del debugger

Tecla	Acción	Cuándo usarla
Clic en margen	Poner/quitar breakpoint	Para marcar dónde parar
Clic derecho en breakpoint	Breakpoint condicional	Cuando el bug solo aparece con ciertos datos
F5	Iniciar / Continuar	Para empezar o saltar al siguiente breakpoint
F10	Step Over	Avanzar línea a línea
F11	Step Into	Entrar dentro de una función
Shift+F11	Step Out	Salir de la función actual
Ctrl+Shift+F5	Reiniciar	Volver a empezar
Shift+F5	Parar	Terminar la sesión de debugging

Paneles

Panel	Qué muestra
Variables	Todas las variables del contexto actual (se actualiza a cada paso)
Watch	Expresiones personalizadas (añadir con +)
Call Stack	Cadena de funciones (clic para cambiar de contexto)

Recursos adicionales

- **Python Tutor:** pythontutor.com — Visualiza la ejecución paso a paso en el navegador
- **VSCoDe Debugging Python:** code.visualstudio.com/docs/python/debugging